Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.9 sur 153 | difficulté 1 / 10

0.9. Tout l'alphabet, avec une seule boucle

Le modulo donne la colonne, la division donne la ligne.

1. Ce qu'il faut comprendre

Ce qui est nouveau ici : `sizeof`, le modulo `%` et la division `/`. Le reste (`for`, `poser`, `ALPHABET[i]`) vient des chapitres précédents.

`sizeof(ALPHABET)` veut dire « la taille de ALPHABET » : le nombre de cases du tableau, **26**. On ne l'écrit plus à la main : si le tableau changeait de taille, la boucle suivrait toute seule. `i < sizeof(ALPHABET)` s'arrête donc après `i = 25`, la dernière lettre.

`i / 20` est une **division entière** : elle garde le quotient et **jette les virgules**. 5 / 20 donne 0 (et non 0,25) ; 19 / 20 donne 0 ; 20 / 20 donne 1 ; 25 / 20 donne 1. Tant que `i` est plus petit que 20, le résultat est 0 : c'est la **ligne 0**. À partir de 20, c'est 1 : la **ligne 1**.

`i % 20` se lit « i modulo 20 » : c'est le **reste** de cette même division. 5 % 20 donne 5 (20 ne rentre pas dans 5, tout reste) ; 19 % 20 donne 19 ; 20 % 20 donne 0 (la division tombe juste, il ne reste rien) ; 25 % 20 donne 5. Le reste monte de 0 à 19 puis **repart à 0** : c'est la **colonne**.

Imagine **20 places par rang**, comme au cinéma : `i / 20` dit combien de rangs **complets** sont déjà remplis (le rang de la lettre), `i % 20` dit combien de places il **reste** une fois ces rangs retirés (sa place dans le rang).

Tour par tour : **i = 0** → colonne 0, ligne 0, **A** · **i = 19** → colonne 19, ligne 0, **T** (la ligne est pleine) · **i = 20** → colonne **0**, ligne **1**, **U** (le reste repart à 0, le quotient passe à 1) · **i = 25** → colonne 5, ligne 1, **Z**.

C'est compliqué à lire : le chapitre suivant montre `poserS`, qui fait ce calcul à notre place.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <reste>` → 0.8.4. #include <reste> — a % b
> - `#include <diviser>` → 0.8.5. #include <diviser> — a / b
> - `#include <ALPHABET>` → 0.1.4. ALPHABET — les lettres de la police
> - `#include <poser>` → 0.1.3. La fonction poser() — poser une tuile

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **affectation (=)** — « x = 5; » : la boîte x prend la valeur 5. Ce n'est pas une égalité : c'est un rangement.
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **for** — Une boucle qui compte : « for (uint8_t i = 0; i < 5; i++) » fait 5 tours, i valant 0, 1, 2, 3 puis 4.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **incrémenter (++)** — « x++ » ou « x += 1 » : x augmente de 1. « x-- » : x diminue de 1.
- **indice** — Le numéro d'une case dans un tableau, entre crochets : cases[3] est la quatrième case (on compte à partir de 0).
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **octet** — Un nombre de 0 à 255 : la plus petite boîte de la mémoire. Une cartouche en a 32 768.
- **opérateurs de calcul** — + plus, - moins, * fois, / divisé par (sans virgule : 7 / 2 donne 3), % le reste de la division (7 % 2 donne 1).
- **opérateurs de comparaison** — == égal, != différent, < plus petit, > plus grand, <= plus petit ou égal, >= plus grand ou égal. Le résultat est vrai ou faux.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **poser()** — Met une tuile dans le décor, sur une case de la grille de 20 × 18 (32 × 32 en tout, avec la partie cachée).
- **uint8_t** — Le genre de nombre de cette console : un entier de 0 à 255 (un octet). Au-delà, il repart à 0.
- **variable** — Une boîte avec un nom, dans la mémoire, qui retient un nombre qu'on peut lire et changer.
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`#include <reste>  // a % b, sauf par 1, 2, 4, 8, 16… écrits en clair`	Demande à la console « reste » (ce que c'est : a % b, sauf par 1, 2, 4, 8, 16… écrits en clair). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « a % b, sauf par 1, 2, 4, 8, 16… écrits en clair ».)
`#include <diviser>  // a / b, sauf par 1, 2, 4, 8, 16… écrits en clair`	Demande à la console « diviser » (ce que c'est : a / b, sauf par 1, 2, 4, 8, 16… écrits en clair). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « a / b, sauf par 1, 2, 4, 8, 16… écrits en clair ».)
`#include <ALPHABET>  // les lettres de la police : ALPHABET[0] est le A`	Demande à la console « ALPHABET » (ce que c'est : les lettres de la police : ALPHABET[0] est le A). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « les lettres de la police : ALPHABET[0] est le A ».)
`#include <poser>  // pose une tuile sur une case du fond`	Demande à la console « poser » (ce que c'est : pose une tuile sur une case du fond). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « pose une tuile sur une case du fond ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`// for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour.`	Un commentaire : une phrase pour celui qui lit (« for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour. »). La console l'ignore.
`// sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25.`	Un commentaire : une phrase pour celui qui lit (« sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25. »). La console l'ignore.
`for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {`	Une boucle qui compte : i part de 0 ; on continue tant que i est plus petit que sizeof(ALPHABET) ; et après chaque tour, i augmente de 1.
`// poser(colonne, ligne, tuile) :`	Un commentaire : une phrase pour celui qui lit (« poser(colonne, ligne, tuile) : »). La console l'ignore.
`//   i % 20      = le RESTE de i divisé par 20 : 0, 1 … 19, puis 0, 1 … 5`	Un commentaire : une phrase pour celui qui lit (« i % 20 = le RESTE de i divisé par 20 : 0, 1 … 19, puis 0, 1 … 5 »). La console l'ignore.
`//                → la colonne, qui repart à 0 après la 19`	Un commentaire : une phrase pour celui qui lit (« → la colonne, qui repart à 0 après la 19 »). La console l'ignore.
`//   i / 20      = le QUOTIENT, sans virgule : 0 jusqu'à i = 19, puis 1`	Un commentaire : une phrase pour celui qui lit (« i / 20 = le QUOTIENT, sans virgule : 0 jusqu'à i = 19, puis 1 »). La console l'ignore.
`//                → la ligne : 0 pour A à T, 1 pour U à Z`	Un commentaire : une phrase pour celui qui lit (« → la ligne : 0 pour A à T, 1 pour U à Z »). La console l'ignore.
`//   ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25`	Un commentaire : une phrase pour celui qui lit (« ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25 »). La console l'ignore.
`poser(i % 20, i / 20, ALPHABET[i]);`	Appel de la fonction poser de la console, avec les arguments i % 20, i / 20, ALPHABET[i]. Ce qu'elle fait : pose une tuile sur la grille du fond — un nom de Tuile, ou un numéro. Un 4e argument est une TAILLE : poser(2, 3, MUR, 5) dessine MUR cinq fois plus grand dans le fond (colonne et ligne en clair).
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`while (true) {`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image().

`image();`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu.
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **938 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage        49 o  ← la console, à l'allumage  — la mise en route de la console
  main()           64 o  ← démarrage  — ta fonction
  VBlank            8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage    28 o  ← image()  — attend l'image suivante
  AttendreVBlank   16 o  ← démarrage  — attend que l'écran soit libre
  AdresseCase      10 o  ← poser()  — trouve l'adresse d'une case de l'écran
  Diviser          17 o  ← poser()  — divise — le processeur ne sait pas le faire
  Reste            10 o  ← poser()  — le reste d'une division
  CopierTuiles     18 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
  EffacerCarte     14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles          704 o  ← CopierTuiles  — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  poser() × 1    39 o sur place, dans main()  + AdresseCase (10 o), Diviser (17 o), Reste (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <reste>     // a % b, sauf par 1, 2, 4, 8, 16… écrits en clair
#include <diviser>   // a / b, sauf par 1, 2, 4, 8, 16… écrits en clair
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <poser>      // pose une tuile sur une case du fond
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
  // for : répète le bloc entre { } ; i part de 0 et augmente de 1 à chaque tour.
  // sizeof(ALPHABET) = 26, la taille du tableau : 26 tours, i va de 0 à 25.
  for (uint8_t i = 0; i < sizeof(ALPHABET); i++) {

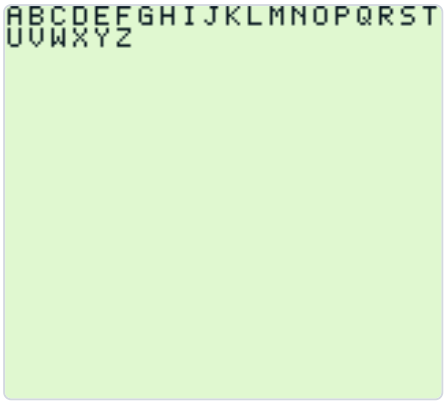
    // poser(colonne, ligne, tuile) :
    //   i % 20      = le RESTE de i divisé par 20 : 0, 1 … 19, puis 0, 1 … 5
    //                 → la colonne, qui repart à 0 après la 19
    //   i / 20      = le QUOTIENT, sans virgule : 0 jusqu'à i = 19, puis 1
    //                 → la ligne : 0 pour A à T, 1 pour U à Z
    //   ALPHABET[i] = la lettre n° i : A pour i = 0, Z pour i = 25
    poser(i % 20, i / 20, ALPHABET[i]);
  }

  while (true) {
    image();
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> A à T sur la ligne 0, U à Z en dessous, comme au 0.8.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la ligne 0 : A à T
- ✓ la ligne 1 : U à Z

Ensuite : **0.9.1. Tout l'alphabet, avec une seule boucle — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++